Applied Cryptography Final Project Report

ML Project-1

Name: Arnav Batra(2022098), Yatin Rathore(2022592)

Abstract

This project investigates whether machine learning can distinguish a reduced-round block cipher from a random permutation by learning relations between paired plaintexts and paired ciphertexts. The chosen primitive is **Speck32/64**, and the task is formulated as a binary classification problem over examples generated with a fixed plaintext XOR difference. We implemented a complete end-to-end pipeline including dataset generation, four input representations, three neural models, result aggregation, and report-quality plotting. The evaluated models are a **Multi-Layer Perceptron (MLP)**, a **1D Convolutional Neural Network (CNN)**, and a **Siamese network**. Experiments were performed for rounds 3 through 8 and aggregated across two training seeds to improve stability. The final results show that **difference-based representations are clearly the most effective**, with the strongest low-round result produced by **Siamese + delta** and the most stable late-round result produced by **MLP + delta**. Under the chosen success criteria, the learned distinguisher remains effective up to about **5 rounds** and then degrades toward chance.

1. Introduction

Classical cryptanalysis often relies on carefully designed relations between differences in plaintexts and the corresponding differences in ciphertexts. Differential cryptanalysis is one of the clearest examples: instead of analyzing isolated encryptions, it studies how structured input differences propagate through the cipher. In recent years, machine learning has been explored as a way of learning such relations automatically rather than deriving them completely by hand.

The central question of this project is therefore:

> Can a neural model learn enough structure from paired plaintext-ciphertext examples to distinguish a reduced-round cipher from a random permutation?

This is exactly the setting described in the assignment. We are not doing full cryptanalysis or key recovery; we are building a **relation-based distinguisher**. If the model can identify a non-random relation in the outputs generated by the cipher, then it is effectively learning signal that should not exist under a uniformly random permutation.

The project was designed to satisfy all required deliverables:

- dataset generation code
- ML model implementation and training code
- experimental results and plots
- a written report

The focus of the implementation is correctness, interpretability, and clear comparison across representations and models.

2. Problem Formulation

Let F be an unknown function that is either:

1. a reduced-round block cipher E_k, or
2. a random permutation π.

For a fixed plaintext XOR difference ΔP, we generate a plaintext pair:

$$(P, P \oplus \Delta P)$$

and obtain the corresponding outputs:

$$(C, C') = (F(P), F(P \oplus \Delta P)).$$

Each sample is labeled as:

- $y = 1$ if the pair was generated by the reduced-round cipher
- $y = 0$ if the pair was generated by the random permutation

The learning task is binary classification:

$$f_{\theta}(P, P \oplus \Delta P, C, C') \rightarrow \{0,1\}$$

where the model parameters θ are learned from data. A successful distinguisher should output higher scores for cipher-generated examples and lower scores for random-permutation examples.

3. Cryptographic Setup

3.1 Chosen Primitive

The implementation uses **Speck32/64**, a lightweight ARX block cipher with:

- block size: 32 bits
- key size: 64 bits
- word size: 16 bits

Speck was chosen for three reasons. First, it is simple enough to implement and verify in a course-project setting. Second, its round structure makes it suitable for reduced-round experiments. Third, it is a standard example of a lightweight block cipher, which fits the assignment's list of suggested primitives.

3.2 Random Baseline

The assignment requires the same setup to be used for both the cipher and the random baseline. For the baseline, we implemented a **lazy random permutation** over the queried domain:

- when a plaintext is queried for the first time, it is assigned a fresh unused 32-bit output
- if the same plaintext appears again, it receives the same output

This design preserves permutation behavior on the sampled dataset without requiring storage of the full (2^{32})-element permutation. For the purposes of the distinguisher experiment, that is exactly the comparison we need.

3.3 Fixed Difference and Rounds

All experiments use the same plaintext XOR difference:

```
Delta_P = 0x0040
```

The reduced-round settings tested are:

```
3, 4, 5, 6, 7, 8 rounds
```

This round sweep allows the project to answer the assignment's key question: **what is the maximum number of rounds for which the learned distinguisher remains effective?**

4. End-to-End Project Workflow

The project pipeline is organized as a sequence of clearly separated stages:

1. implement the cipher and the random permutation baseline
2. generate labeled paired-plaintext datasets for each round count
3. convert raw tuples into several input representations
4. train multiple ML models on each representation
5. evaluate on held-out test data
6. aggregate results across training seeds
7. generate plots and summary tables for the final report

This structure keeps the project modular. The cryptographic setup, dataset generation, representations, models, and reporting are implemented in separate modules so that experiments remain reproducible and easy to inspect.

5. Dataset Generation

5.1 Sampling Procedure

For every round count, the dataset generation code performs the following steps:

1. sample random 32-bit plaintexts (P)
2. compute paired plaintexts (P' = P \oplus \Delta P)
3. sample a binary label (y)
4. if (y = 1), generate ((C, C')) using reduced-round Speck
5. if (y = 0), generate ((C, C')) using the random permutation oracle
6. save the arrays in compressed `.npz` format

The saved bundle contains:

- `p`
- `p_pair`
- `c`
- `c_pair`
- `labels`

This directly satisfies the dataset-generation deliverable in the assignment.

5.2 Dataset Sizes

For each tested round:

Split	Samples
Train	12,000
Validation	3,000
Test	3,000

So each round uses **18,000 samples**, and the full experiment uses **108,000 samples** across all six round settings.

5.3 Quality Checks

The generated data was checked for practical correctness:

- label balance stays close to 50:50
- the random permutation remains consistent for repeated queries
- sampled ciphertext outputs in the test set do not collide accidentally
- split generation is reproducible from fixed seeds

The label-balance summary is shown below.

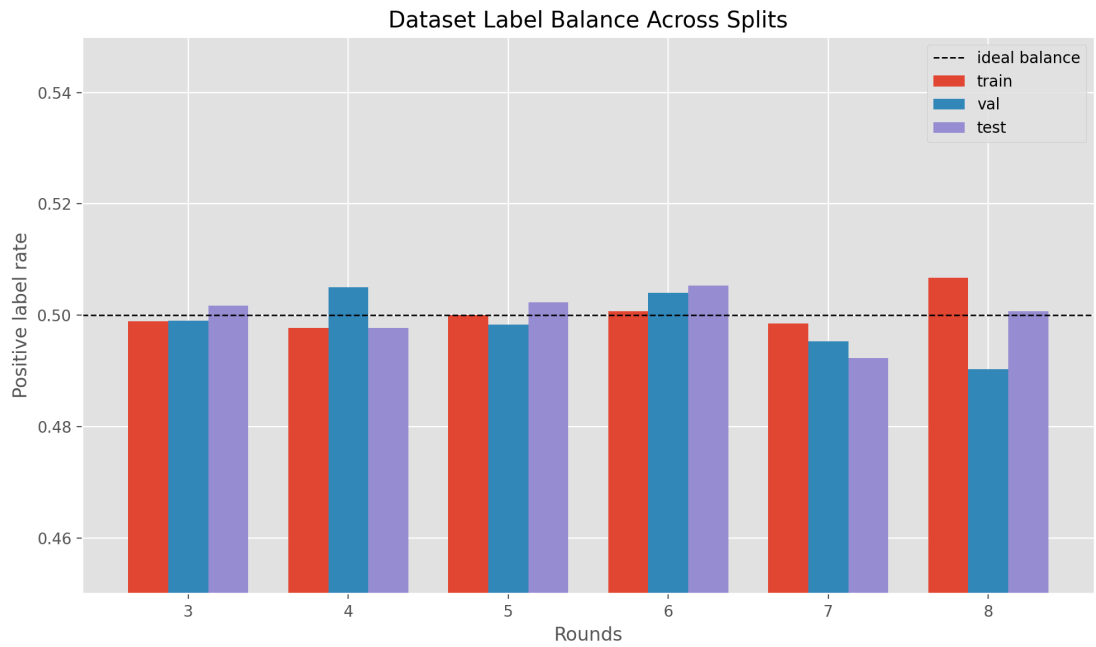

Figure 1. Positive label rate across rounds and splits. The observed rates remain close to 0.5, indicating good class balance.

From `dataset_balance.csv`, the smallest positive rate is `0.4903` and the largest is `0.5067`, so the datasets are well balanced.

6. Input Representations

The assignment explicitly asks students to experiment with multiple representations and discuss their impact. To address that requirement, four representations were implemented.

6.1 `delta`

[\Delta C = C \oplus C']

This is the most direct representation for differential-style learning. Instead of giving the model two ciphertexts and asking it to infer the useful comparison on its own, it provides the relation itself. This turned out to be the strongest representation overall.

6.2 `concat`

[C \Vert C']

This representation preserves the raw pair. It is more general than `delta`, but the model must learn both the comparison rule and the final classification decision. As expected, it performs worse than `delta`.

6.3 `joint`

[(P, C, P \oplus \Delta P, C')]

This representation gives the model both inputs and outputs. In theory, it contains more information. In practice, it creates a much larger feature vector and makes optimization harder. The final results show that simply giving more raw information is not enough if the representation is not aligned with the cryptanalytic relation of interest.

6.4 `delta_stats`

This representation augments the raw difference bits with simple handcrafted features:

- total Hamming weight of (\Delta C)
- left-half Hamming weight
- right-half Hamming weight
- nibble-level Hamming weights
- left-right weight gap

The purpose of this design is to test whether coarse statistical summaries can help the models when the differential signal becomes weaker. The final experiments show that `delta_stats` is not the best overall representation, but it does improve some mid-round behavior, especially at round 4.

7. ML Models and Training

7.1 Implemented Models

The assignment requires implementation and comparison of at least three models. The project includes:

- **MLP**
- **CNN**
- **Siamese network**

MLP

The MLP takes flattened bit vectors and passes them through fully connected layers with ReLU activations, batch normalization, and dropout. It is a strong baseline for fixed-length binary features.

CNN

The CNN treats the bit representation as a one-dimensional signal and applies convolutions to capture local bit patterns and short-range structure. This is especially useful when neighboring bits may contain meaningful relations.

Siamese Network

The Siamese model splits the input into two halves, processes them through a shared encoder, and combines the embeddings using direct concatenation, absolute difference, and elementwise product. This architecture is natural for pair-based data and produced the strongest low-round performance.

7.2 Training Protocol

All experiments use the same basic training setup:

- optimizer: Adam
- learning rate: `0.001`
- batch size: `256`
- weight decay: `1e-5`
- maximum epochs: `16`
- early stopping patience: `4`
- scheduler: `ReduceLROnPlateau`
- device: CPU

To reduce the chance that the final result is caused by a lucky initialization, each configuration is trained with **two seeds**: `1337` and `2027`. The final tables and plots use the **mean** values across seeds and keep track of standard deviations.

7.3 Evaluation Metrics

Three evaluation metrics are emphasized:

- **Accuracy**
- **Balanced accuracy**
- **ROC-AUC**

ROC-AUC is important because it evaluates ranking quality across thresholds. Balanced accuracy is important because a weak model can sometimes achieve misleading raw accuracy by favoring one class. For this project, balanced accuracy and ROC-AUC are the most meaningful success metrics.

To define the **maximum effective round**, a configuration must satisfy:

- mean ROC-AUC (\ge 0.60)

- mean balanced accuracy (\ge 0.60)
- both seeds must satisfy the thresholds

This makes the final conclusion stricter and more reliable than a single lucky run.

8. Experimental Results

8.1 Round-Wise Behavior

The clearest trend in the project is that distinguishability is strong at low rounds and then steadily collapses as rounds increase.

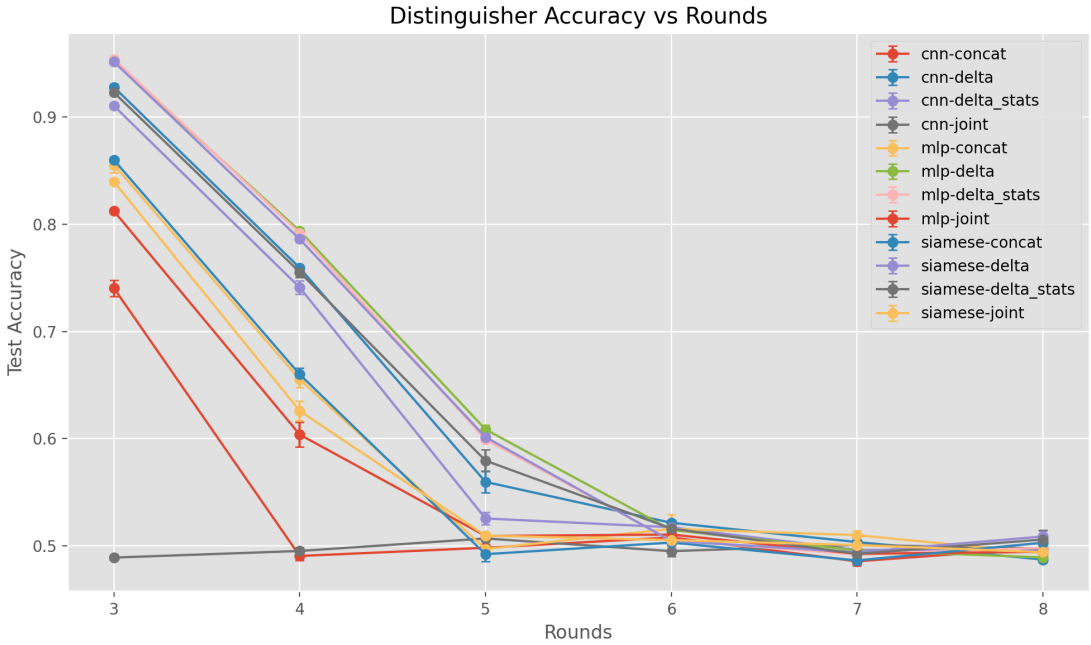

Figure 2. Test accuracy across all model-representation combinations.

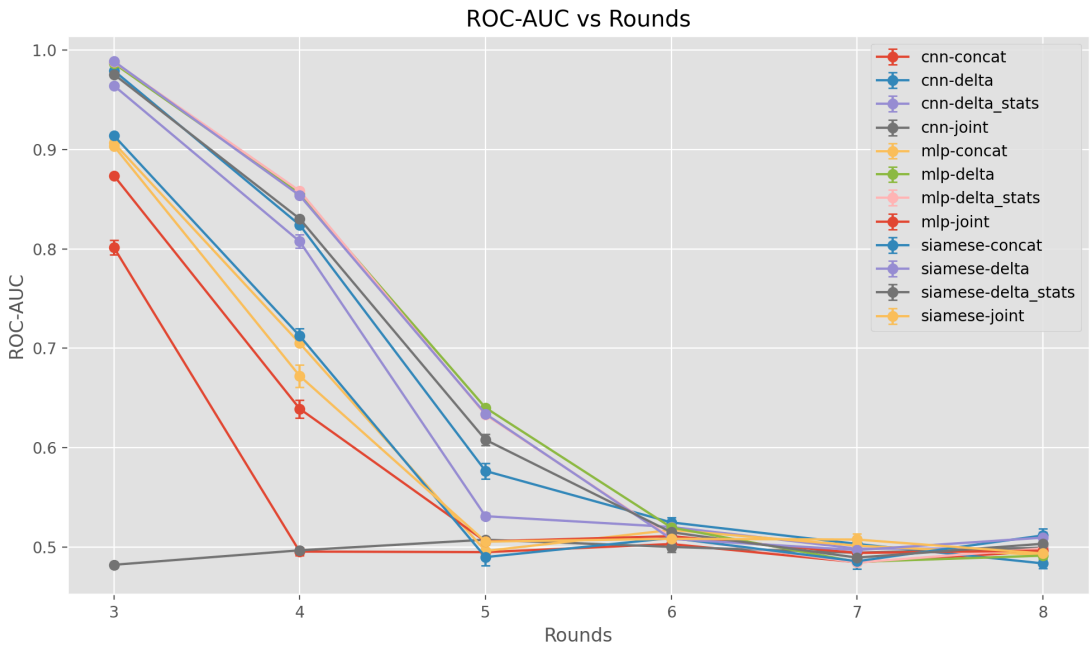

Figure 3. ROC-AUC across rounds. Difference-based representations dominate the low-round regime.

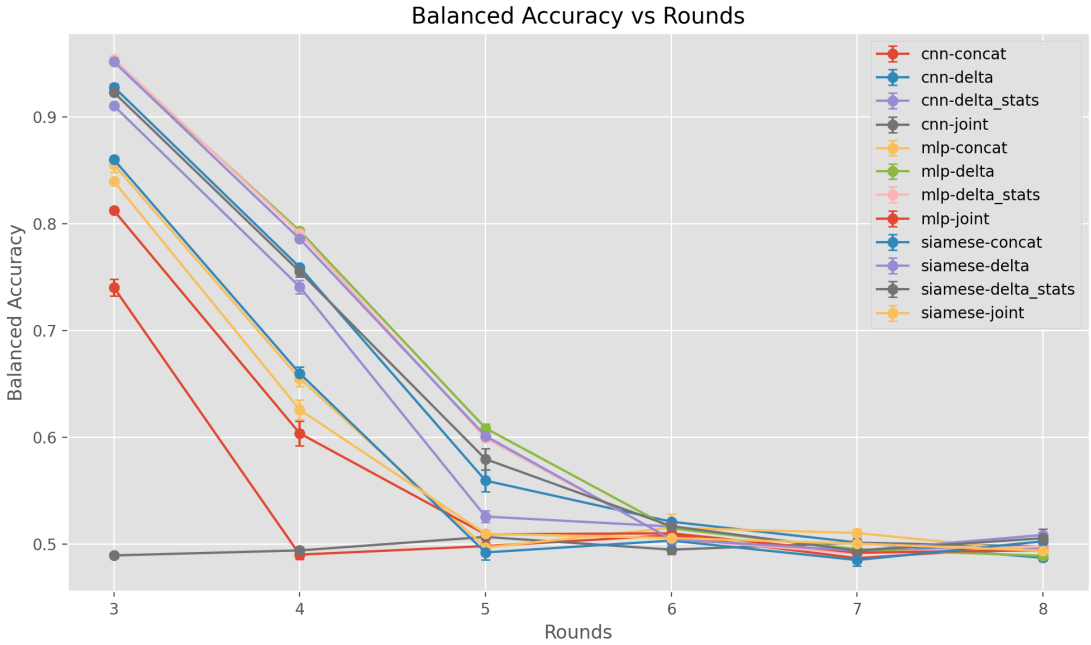

Figure 4. Balanced accuracy confirms that meaningful signal is largely gone after round 5.

These plots tell a consistent story:

- rounds 3 and 4 are clearly distinguishable
- round 5 still has weak but meaningful signal
- rounds 6, 7, and 8 are essentially near chance for almost all settings

8.2 Best Result at Each Round

The best configuration at each round, chosen by ROC-AUC, is:

Round	Best Model	Representation	Accuracy	Balanced Accuracy	ROC-AUC
3	Siamese	delta	0.9517	0.9517	0.9888
4	MLP	delta_stats	0.7917	0.7912	0.8584
5	MLP	delta	0.6085	0.6086	0.6402
6	CNN	delta	0.5213	0.5209	0.5248
7	Siamese	joint	0.5005	0.5004	0.5076
8	Siamese	concat	0.5027	0.5026	0.5117

This table is important because it directly answers how the task changes as the cipher becomes harder:

- **Round 3:** the relation is very strong and easy to learn
- **Round 4:** the signal is weaker but still very clear
- **Round 5:** the signal is now weak and close to the edge of usefulness
- **Rounds 6–8:** the models are close to random guessing

The corresponding saved file is `best_by_round.csv`.

8.3 Maximum Effective Round

The assignment specifically asks for the **maximum number of rounds for which a relation-based distinguisher remains effective**. That result is summarized below.

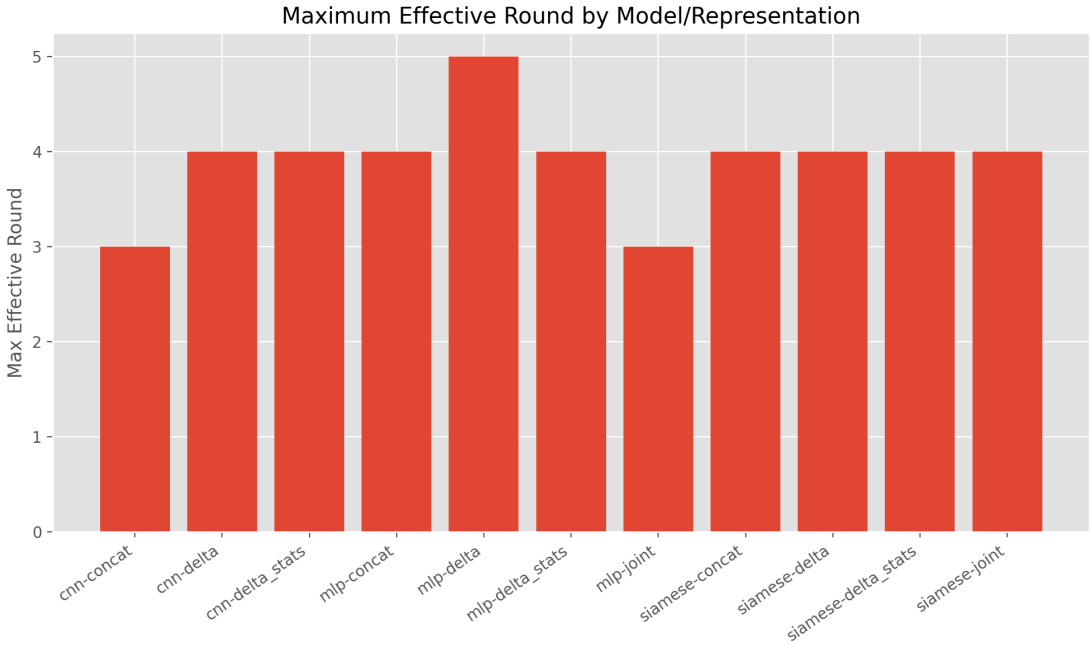

Figure 5. Maximum effective round under the chosen ROC-AUC and balanced-accuracy thresholds.

Model	Representation	Maximum Effective Round
CNN	concat	3
CNN	delta	4
CNN	delta_stats	4
MLP	concat	4
MLP	delta	5
MLP	delta_stats	4
MLP	joint	3
Siamese	concat	4
Siamese	delta	4
Siamese	delta_stats	4
Siamese	joint	4

The strongest final statement from this table is:

> **MLP + delta** is the only configuration in this project that remains effective through **5 rounds** under the strict multi-seed rule.

8.4 Overall Model and Representation Comparison

The next question is whether model choice or representation choice mattered more.

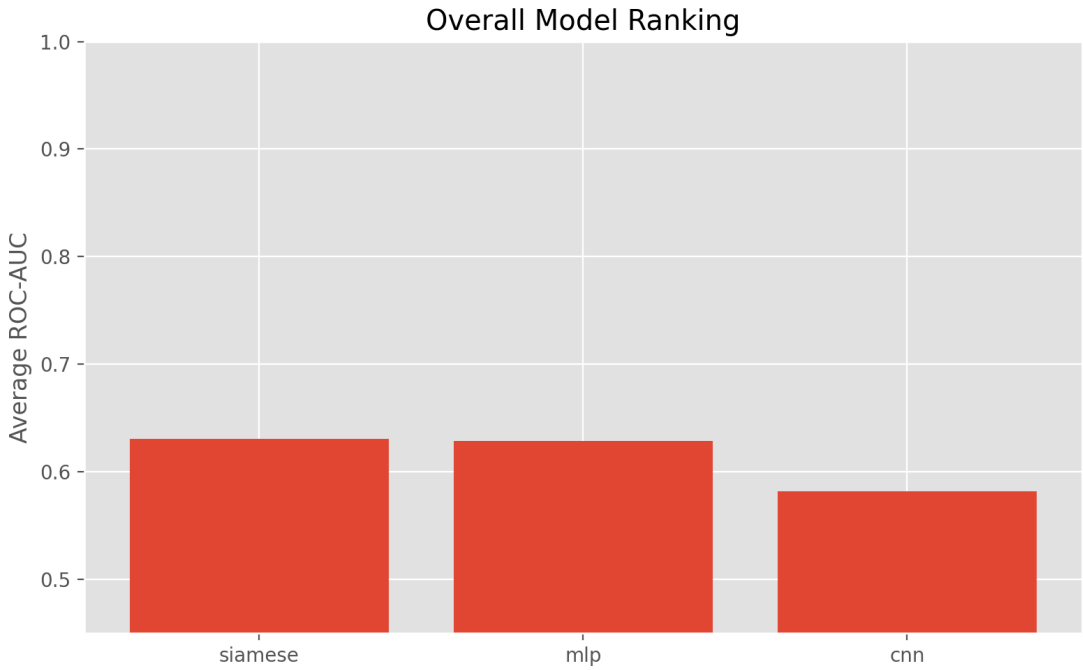

Figure 6. Average ROC-AUC for each model across all rounds and representations.

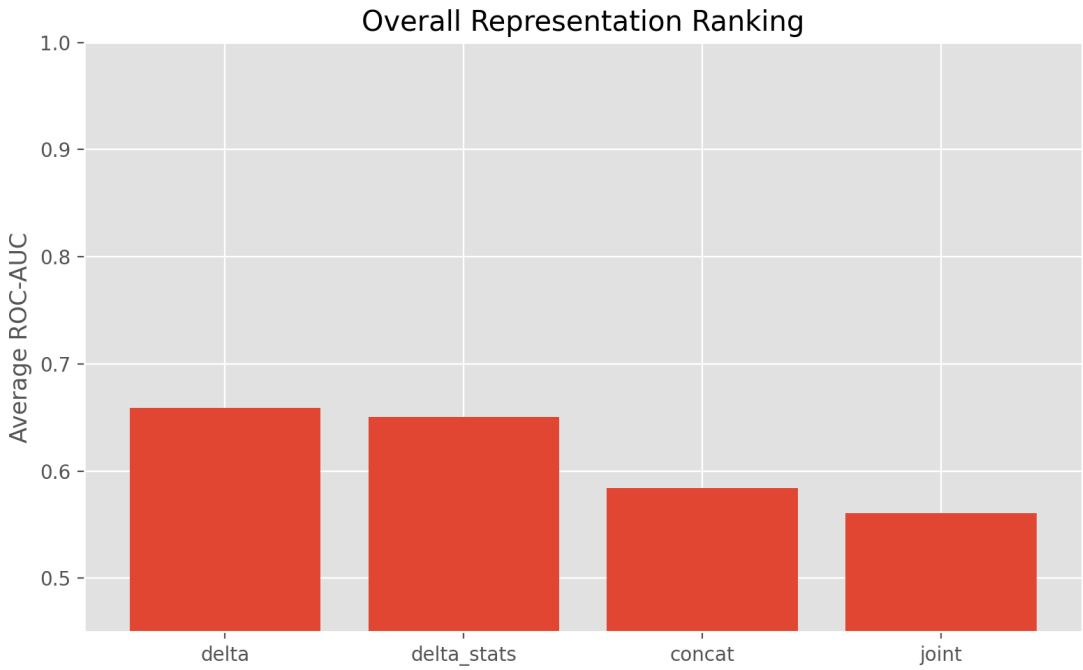

Figure 7. Average ROC-AUC for each representation across all rounds and models.

The saved rankings show:

- representation means:
- `delta`: `0.6590`
- `delta_stats`: `0.6508`
- `concat`: `0.5843`
- `joint`: `0.5606`
- model means:
- `siamese`: `0.6303`
- `mlp`: `0.6287`
- `cnn`: `0.5820`

This comparison is one of the most important findings in the project. It shows that **representation choice mattered more than model choice**. The best two representations are both difference-based, and they outperform the more naive encodings by a noticeable margin. Among models, Siamese and MLP are very close overall, while CNN is somewhat weaker in this setup.

9. Discussion and Interpretation

9.1 Why `delta` Worked Best

The best-performing representation, `delta`, is the one most closely aligned with the intuition behind differential cryptanalysis. If the cipher induces structured relations between paired plaintexts and paired ciphertexts, then those relations are most naturally expressed in the ciphertext difference. By supplying (\Delta C) directly, the model starts from the right relational view of the data instead of having to discover that view from raw pairs.

9.2 Why `delta_stats` Helped

The `delta_stats` representation adds lightweight handcrafted features on top of the raw bit pattern. It did not overtake `delta` as the best overall feature set, but it produced the best result at round 4. This suggests that even simple statistics such as Hamming weights can add useful signal when the raw relation is beginning to weaken.

9.3 Why `joint` Underperformed

At first glance, `joint` seems like it should perform well because it contains the most information. However, more information does not automatically mean easier learning. In this case, the representation becomes much larger, less focused, and harder to optimize. The models appear to benefit more from structured relational features than from an enlarged raw input.

9.4 What the Results Mean Cryptanalytically

The project does **not** claim that Speck32/64 is insecure in general, nor does it claim that no distinguisher exists beyond 5 rounds. Instead, the conclusion is narrower and more precise:

- for the chosen plaintext difference `0x0040`
- for the selected representations and model family
- for the available sample budget
- under the strict two-seed stability rule

the learned distinguisher remains effective up to about **5 rounds** and then loses reliable signal.

That is exactly the kind of conclusion expected from a relation-based neural distinguisher project.

10. Conclusion

This project implemented a complete machine-learning-based cryptanalysis workflow for distinguishing reduced-round Speck32/64 from a random permutation. The deliverables requested in the assignment were all covered:

- dataset generation code
- implementations of at least three ML models
- training and experiment code
- plots and result tables
- a written report

The main empirical conclusions are:

- low-round Speck is highly distinguishable from a random permutation
- difference-based representations are the strongest
- the best low-round result is **Siamese + delta**
- the most stable maximum-round result is **MLP + delta**
- effective distinguishability persists through about **5 rounds**

11. How to Run

Install the required packages:

```
pip install -r requirements.txt
```

Generate the datasets:

```
python main.py generate-dataset --config configs/default.yaml
```

Run the full experiment sweep and generate the final plots and CSV summaries:

```
python main.py run-experiments --config configs/default.yaml
```

After running the experiments, the final outputs will be available in:

- `results/default/` for plots and result tables
- `report/` for the written report

12. References

[1] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. *The SIMON and SPECK families of lightweight block ciphers*. Technical report, Cryptology ePrint Archive, 2013.

[2] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. *Mutual information neural estimation*. In *International Conference on Machine Learning*, pages 531-540. PMLR, 2018.

[3] Aron Gohr. *Improving attacks on round-reduced Speck32/64 using deep learning*. In *Annual International Cryptology Conference*, pages 150-179. Springer, 2019.

[4] Used AI for ideation, learning and understanding purpose.